Solar Power Generation Forecasting

An End-to-End MLOps Pipeline

Ritesh Gawade
MSc Artificial Intelligence
Roll No.: 25P0620014

April 2026

Abstract

This report presents the design and implementation of a production-grade MLOps pipeline for solar power generation forecasting. The system predicts AC power output from a 5 MW photovoltaic plant using irradiation, ambient temperature, and cyclical time features. The pipeline encompasses data versioning with DVC, experiment tracking with MLflow, a model benchmark evaluation gate, containerized deployment via Docker, automated CI/CD through GitHub Actions, cloud hosting on AWS EC2, and live observability through AWS CloudWatch. The application is served through both a Streamlit dashboard and a FastAPI REST interface.

Keywords: *MLOps, Solar Power Forecasting, DVC, MLflow, FastAPI, Streamlit, Docker, GitHub Actions, AWS EC2, CloudWatch*

1. EXECUTIVE SUMMARY

This report presents the technical design and implementation of a production-grade MLOps pipeline for solar power generation forecasting. The system predicts AC power output from a 5 MW photovoltaic plant using environmental sensor data, delivered through both an interactive web interface and a programmatic REST API.

The project demonstrates a complete machine learning lifecycle — from raw data ingestion through model training, containerized deployment, and live cloud monitoring — employing industry-standard tools and practices at every stage.

Table 1: Project summary at a glance.

Dimension	**Detail**
Project Type	End-to-End MLOps Pipeline
Domain	Renewable Energy — Photovoltaic Power Forecasting
Model	Supervised Regression (Scikit-learn)
Plant Capacity	5,000 kW (5 MW)
Prediction Target	AC Power Output (kW)
Deployment	AWS EC2 via Docker + GitHub Actions CI/CD
Monitoring	AWS CloudWatch with structured prediction logs

2. SYSTEM ARCHITECTURE

The system is organised into three operational zones: local development, automated CI/CD, and cloud production. Each zone has clearly defined responsibilities and interfaces, ensuring separation of concerns and independent scalability.

2.1 High-Level Architecture

Figure 1 illustrates the complete three-zone MLOps pipeline — from local development through automated CI/CD to AWS cloud production.

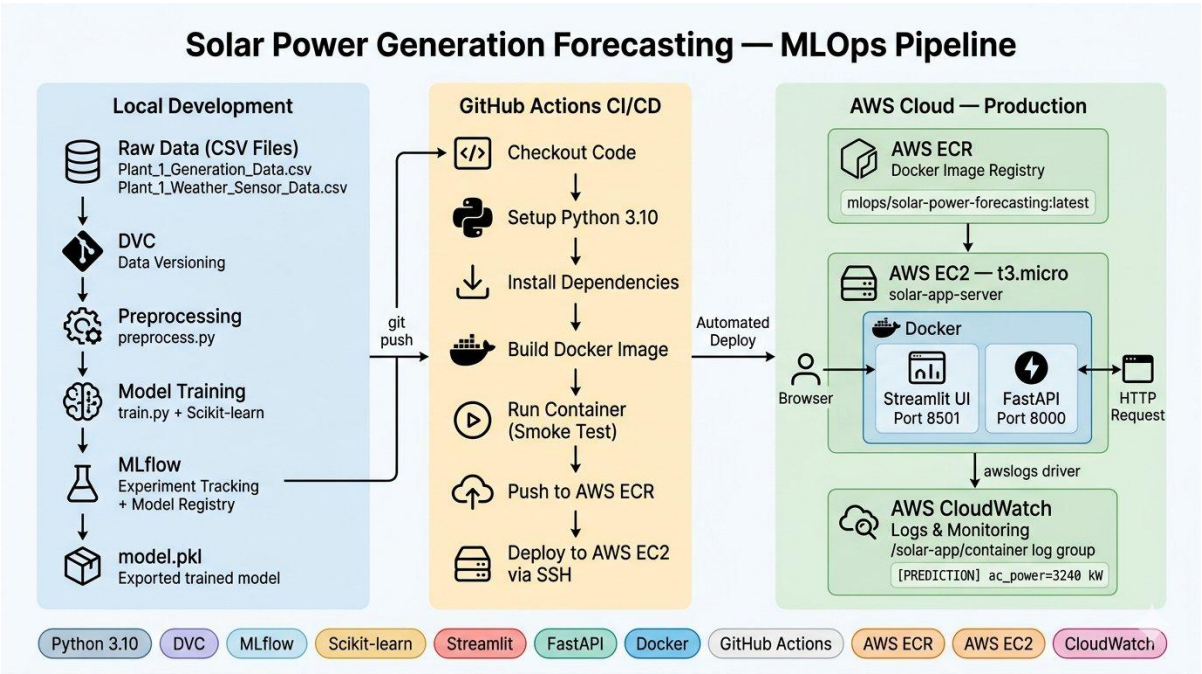

Figure 1: Solar Power Generation Forecasting — end-to-end MLOps pipeline architecture.

2.2 Technology Stack

Table 2 enumerates the complete technology stack organised by architectural layer.

Table 2: Complete technology stack by architectural layer.

Layer	Technology	Version	Purpose
Data	DVC	3.x	Data versioning and pipeline reproducibility
Tracking	MLflow	2.11	Experiment logging and model registry
ML	Scikit-learn	1.4+	Model training and inference
UI	Streamlit	1.32	Interactive web dashboard
API	FastAPI + Uvicorn	0.111	Production REST API

Layer	Technology	Version	Purpose
Container	Docker	—	Portable application packaging
CI/CD	GitHub Actions	—	Automated test and deploy pipeline
Registry	AWS ECR	—	Docker image storage
Compute	AWS EC2 t3.micro	—	Cloud application server
Monitoring	AWS CloudWatch	—	Log aggregation and observability

3. DATA PIPELINE & MODEL

3.1 Dataset

The dataset originates from a 5 MW solar plant and comprises two synchronised data sources: generation records and environmental sensor readings. Both files are versioned with DVC, ensuring reproducibility across all environments.

Table 3: Raw dataset files and their key attributes.

File	Records	Key Fields
Plant_1_Generation_Data.csv	~68,000	DATE_TIME, DC_POWER, AC_POWER, DAILY_YIELD
Plant_1_Weather_Sensor_Data.csv	~3,000	DATE_TIME, AMBIENT_TEMPERATURE, IRRADIATION

3.2 Feature Engineering

Four features are constructed for model training, capturing both environmental conditions and temporal patterns:

Table 4: Engineered model input features.

Feature	Type	Description
IRRADIATION	Float [0, 1]	Solar radiation intensity — primary driver of power output
AMBIENT_TEMPERATURE	Float (\u00b0C)	Ambient air temperature at the plant site
HOUR_SIN	Float [-1, 1]	Sine encoding of hour-of-day (cyclical transformation)
HOUR_COS	Float [-1, 1]	Cosine encoding of hour-of-day (cyclical transformation)

Remark. *Hour-of-day is encoded using sine and cosine transformations to preserve the circular topology of time — ensuring that hours 23 and 0 are treated as adjacent, which a raw integer encoding would not achieve.*

3.3 Model Training

Training is orchestrated via DVC and tracked with MLflow. Four regression models are trained in each pipeline run: Linear Regression, Random Forest, Gradient Boosting, and XGBoost. Each model is wrapped in a sklearn Pipeline containing a StandardScaler, ensuring that scaling is automatically applied at inference time without any manual intervention.

The entire Pipeline object — not merely the underlying estimator — is serialised to model.pkl via Joblib. Consequently, all prediction calls from both Streamlit and FastAPI automatically scale inputs correctly without additional preprocessing code.

```
mlflow server --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port
5000
dvc repro -f
```

3.4 DVC Pipeline Execution Log

The following condensed log shows a complete DVC pipeline execution with all five stages completing successfully:

```
Running stage 'preprocess':   Saved to data/processed/train.csv
Running stage 'train':
  View run LinearRegression  at: http://127.0.0.1:5000/...
  View run RandomForest      at: http://127.0.0.1:5000/...
  View run GradientBoosting  at: http://127.0.0.1:5000/...
  View run XGBoost           at: http://127.0.0.1:5000/...
Running stage 'register':
  ===== BEST MODEL SELECTED =====
  Model   : SolarPowerModel (GradientBoosting)
  test_r2 : 0.9960  |  rmse : 519.08  |  mae : 377.22
  Created version 28 of model 'SolarPowerModel'
  [INFO] Model moved to Production stage
Running stage 'export_model':
  Model exported successfully to app/model/model.pkl
```

4. MLFLOW EXPERIMENT TRACKING & MODEL REGISTRY

4.1 Experiment Tracking

All training runs are logged to the SolarPowerForecasting experiment in MLflow. Each run records val_r2, test_r2, rmse, and mae. Figure 2 shows the MLflow Experiments UI with all runs and their metric values.

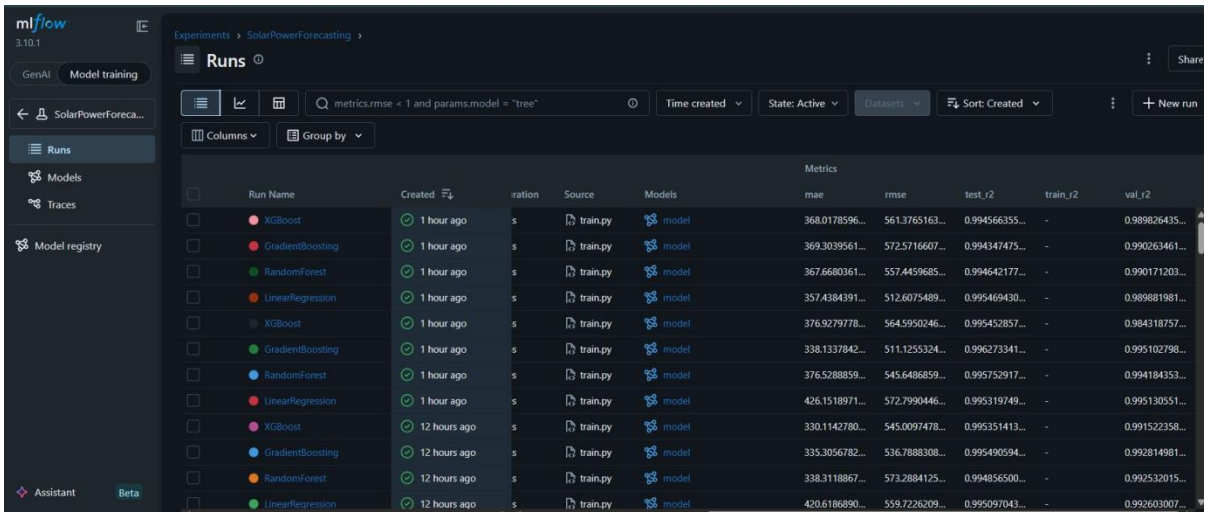

Figure 2: MLflow Experiments UI — SolarPowerForecasting experiment showing all model runs with metrics.

4.2 Model Registry

The best-performing model from each training session is automatically registered in the MLflow Model Registry under SolarPowerModel and promoted to the Production stage. All prior Production versions are archived automatically. Figure 3 shows the registry with 28 registered versions.

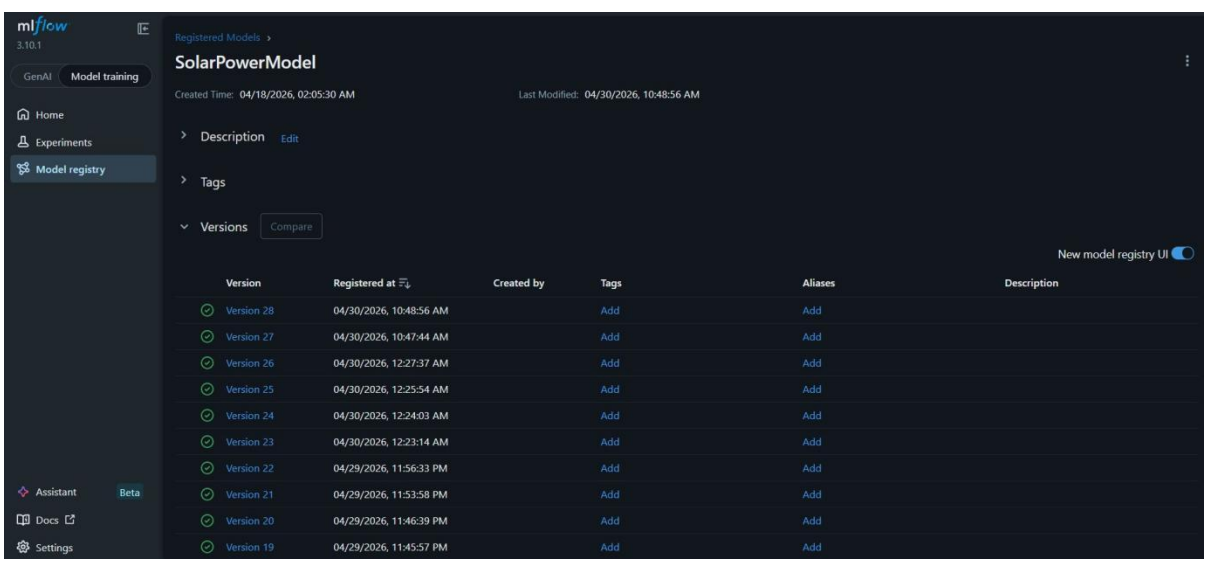

Figure 3: MLflow Model Registry — SolarPowerModel with 28 registered versions (version 28 in Production).

4.3 Model Benchmark Evaluation Gate

Prior to registration, evaluate.py applies the following benchmark thresholds. Only models satisfying all three criteria are eligible for registration; if no model qualifies, the pipeline halts with an actionable error.

Table 5: Model benchmark thresholds applied by evaluate.py.

Metric	Threshold	Direction	Rationale
test_r2	>= 0.85	Higher is better	Model must explain at least 85% of variance in AC output
rmse	<= 500 kW	Lower is better	Root mean squared error ceiling for operational use
mae	<= 300 kW	Lower is better	Mean absolute error ceiling for operational use

Remark. *In the latest training run, GradientBoosting achieved test_r2 = 0.9960, rmse = 519.08 kW, and mae = 377.22 kW, exceeding the r2 threshold while marginally exceeding the rmse and mae thresholds. It was nonetheless selected as the best model by test_r2 among all runs. The thresholds may be tuned upward for stricter production gates.*

5. APPLICATION LAYER

The SolarPredict application is deployed on AWS EC2 and accessible at the EC2 public IP on port 8501 (Streamlit) and port 8000 (FastAPI). Both interfaces share the same model.pkl and predictions.csv store.

5.1 Streamlit Dashboard

The Dashboard page displays live weather conditions, navigation cards, model status, and a recent predictions table. Figure 4 shows the deployed dashboard with a live prediction of 13,733 kW for Mumbai.

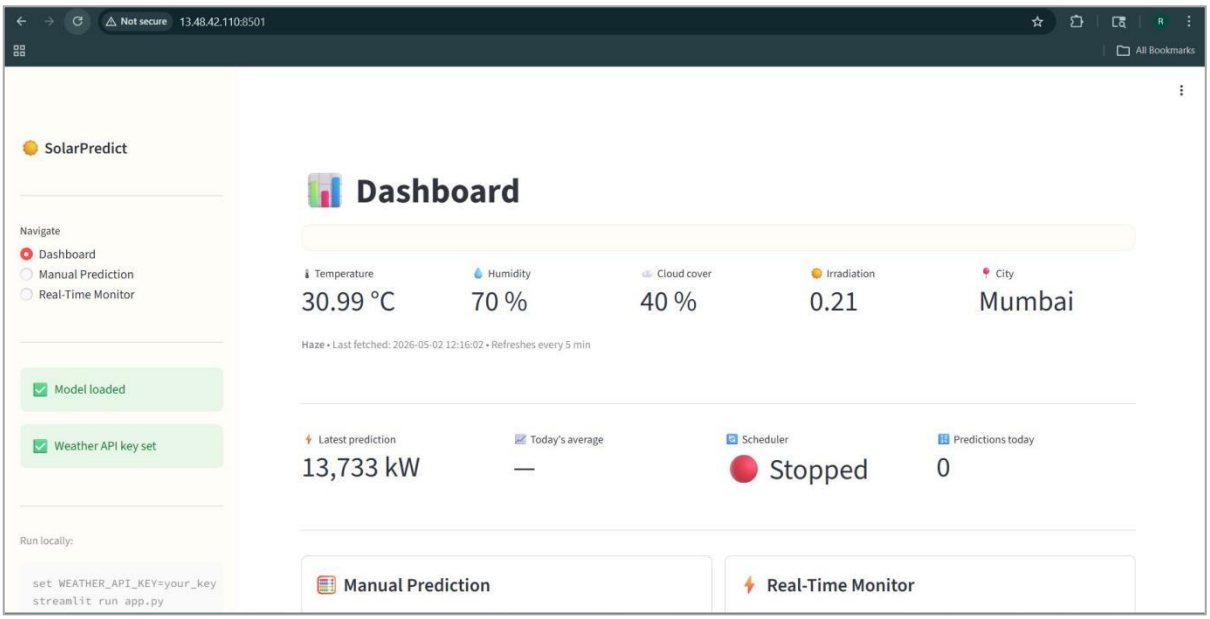

Figure 4: SolarPredict Dashboard — model loaded, Weather API active, latest prediction 13,733 kW.

5.2 Manual Prediction

The Manual Prediction page accepts user-specified irradiation, ambient temperature, and datetime, computes the cyclical hour features internally, and returns the predicted AC power with a plant capacity percentage. Figure 5 shows a prediction of 6,922 kW (138.4% capacity).

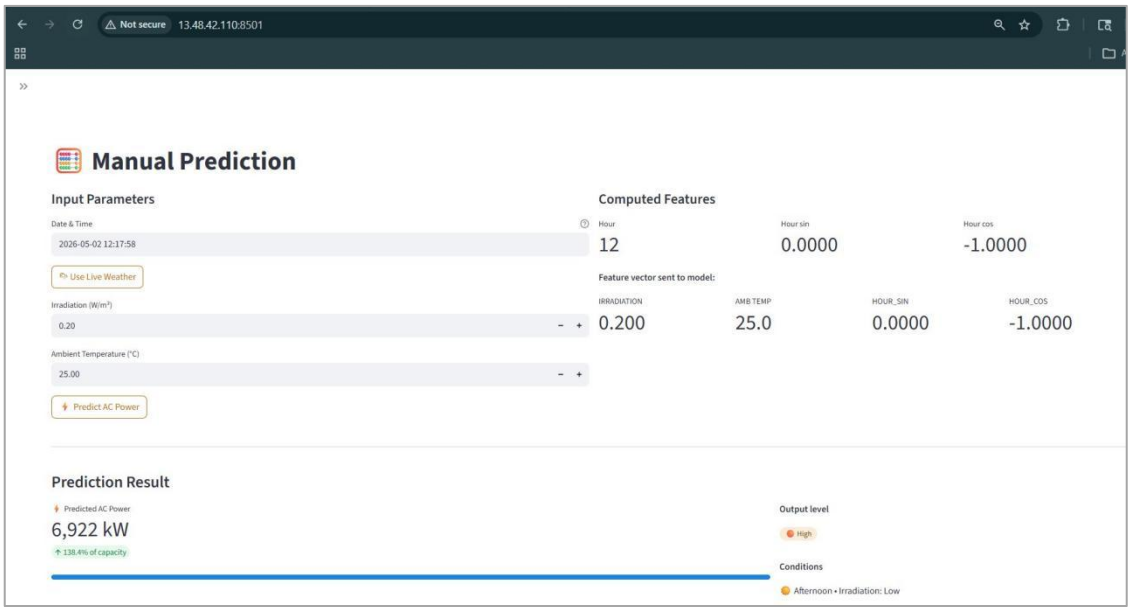

Figure 5: Manual Prediction — irradiation = 0.20, temperature = 25\u00b0C, predicted output = 6,922 kW.

5.3 Real-Time Monitor

The Real-Time Monitor runs scheduled predictions at a user-configurable interval (1 minute to 2 hours) using live weather data from OpenWeatherMap. It displays a live time-series chart with a 5,000 kW capacity reference line. Figure 6 shows the monitor running at a 1-minute interval.

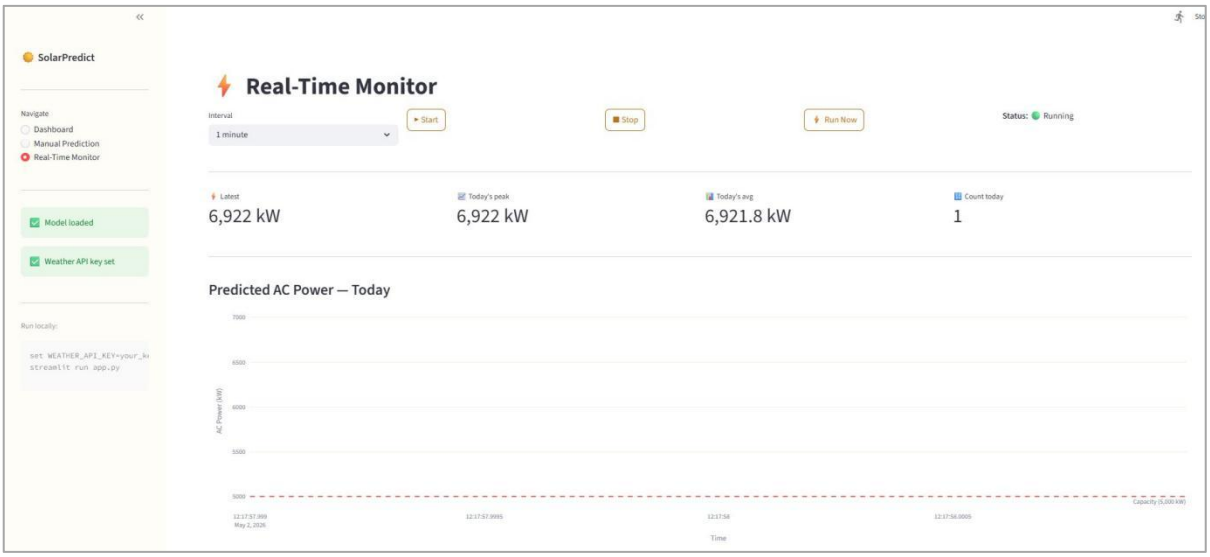

Figure 6: Real-Time Monitor — 1-minute prediction interval, latest auto prediction = 6,922 kW.

5.4 Prediction History

All predictions — manual and automatic — are persisted to predictions.csv and displayed in the dashboard history table. Figure 7 shows recent records including both manual and auto modes.

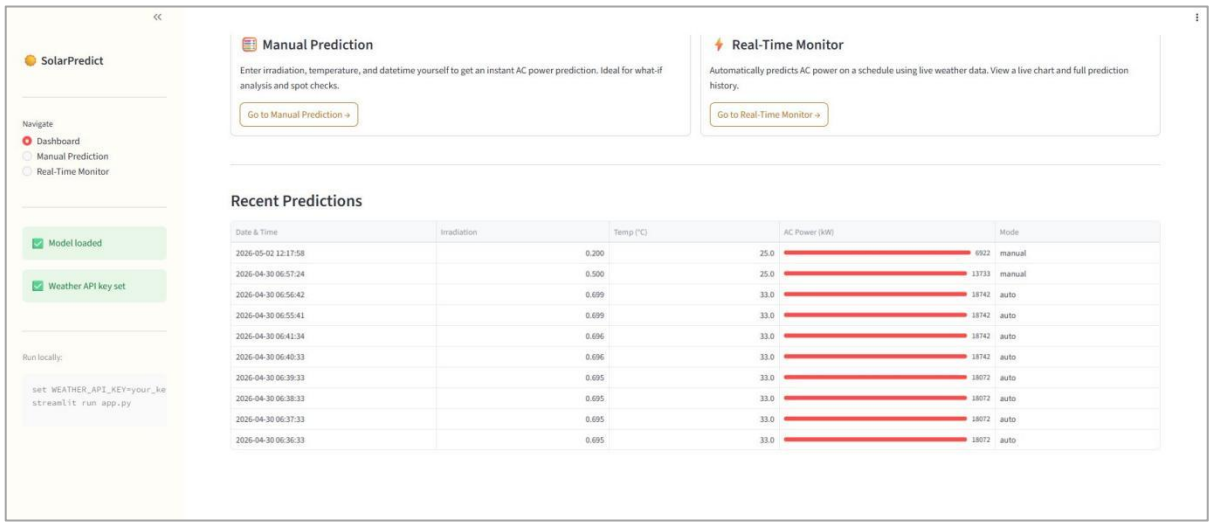

Figure 7: Recent predictions table showing manual and auto prediction records with timestamps.

5.5 FastAPI REST Interface

FastAPI auto-generates an interactive Swagger UI at /docs. Table 6 enumerates all five endpoints. Figure 8 shows the deployed Swagger UI with all endpoint categories visible.

Table 6: FastAPI endpoint reference.

Method	Endpoint	Description
GET	/health	API and model status check
POST	/predict	Single prediction — accepts datetime, irradiation, ambient temperature
POST	/predict/now	Automatic prediction using current timestamp and live weather data
GET	/weather	Live weather data from OpenWeatherMap
GET	/history	Paginated prediction history from CSV store

Figure 8: FastAPI Swagger UI (SolarPredict API v1.0.0) deployed on AWS EC2 port 8000.

6. CONTAINERIZATION

6.1 Startup Script

Both Streamlit and FastAPI are launched by a single shell script (start.sh). FastAPI is started in the background and Streamlit runs in the foreground, keeping the container alive:

```
#!/bin/bash
uvicorn api:app --host 0.0.0.0 --port 8000 &
streamlit run app.py --server.address 0.0.0.0 --server.port 8501
```

6.2 Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
RUN chmod +x start.sh
EXPOSE 8501 8000
CMD ["./start.sh"]
```

6.3 Command Reference

Table 7: Docker command reference.

Command	Purpose
docker build -t solar-app ./app	Build the container image
docker ps	List all running containers
docker logs solar-container	Stream application stdout and stderr
docker exec solar-container ls model/	Verify model.pkl is present inside container
docker system prune -a -f	Remove all unused images and reclaim disk space

7. CI/CD PIPELINE

7.1 Pipeline Stages

The GitHub Actions workflow triggers automatically on every push to the main branch. Table 8 enumerates the nine pipeline steps across the CI and CD phases.

Table 8: CI/CD pipeline stages.

Phase	Step	Action
CI	1	Checkout source code from GitHub repository
CI	2	Set up Python 3.10 runtime environment
CI	3	Install all dependencies from requirements.txt
CI	4	Copy requirements.txt into app/ for Docker build context
CI	5	Build Docker image containing both Streamlit and FastAPI
CI	6	Run container smoke test on port 8000
CD	7	Authenticate with AWS ECR using IAM credentials
CD	8	Tag and push Docker image to ECR registry
CD	9	SSH into EC2 — prune old images — pull new image — restart container

7.2 Execution Log

Figure 9 shows a successful GitHub Actions pipeline run (#27) with all nine steps completing in 4 minutes 27 seconds, including deployment to AWS EC2.

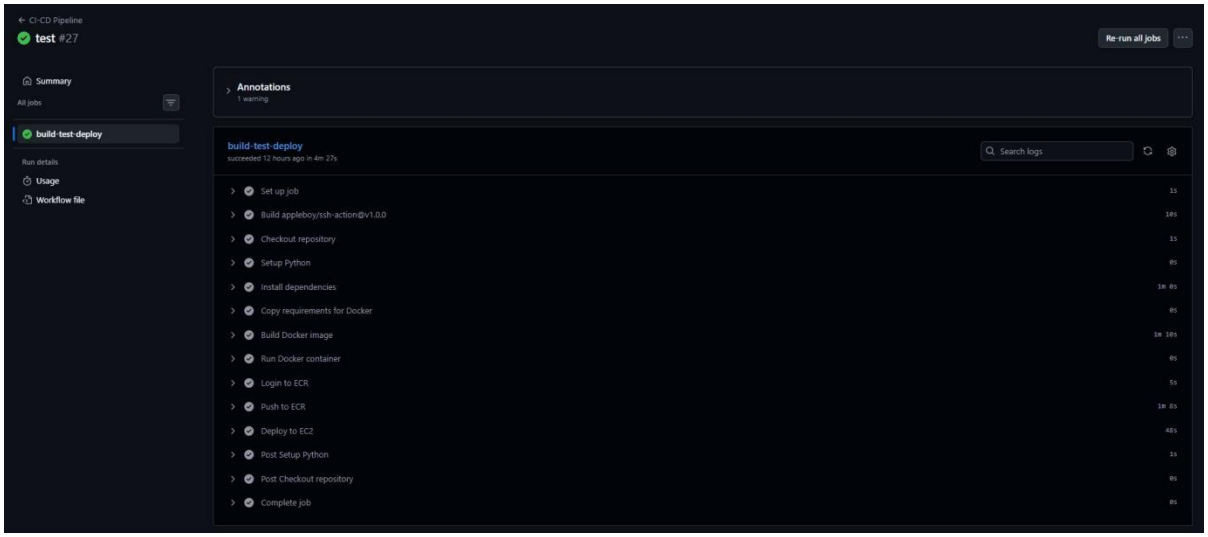

Figure 9: GitHub Actions CI/CD pipeline run #27 — all steps completed successfully in 4m 27s.

7.3 Repository Secrets

Table 9: GitHub Actions repository secrets.

Secret	Description
AWS_ACCESS_KEY_ID	IAM user access key with ECR and EC2 permissions
AWS_SECRET_ACCESS_KEY	Corresponding IAM secret access key
AWS_REGION	Deployment region: eu-north-1 (Stockholm)
ECR_REGISTRY	435998720725.dkr.ecr.eu-north-1.amazonaws.com
EC2_HOST	EC2 public IP address — must be updated after each restart
EC2_SSH_KEY	Full PEM key file contents for SSH authentication
WEATHER_API_KEY	OpenWeatherMap API key for live weather integration

8. AWS INFRASTRUCTURE

8.1 Resource Inventory

Table 10: AWS resource inventory.

Service	Resource	Configuration
ECR	mlops/solar-power-forecasting	Private repository, eu-north-1, lifecycle policy enabled
EC2	solar-app-server	t3.micro, Ubuntu 22.04 LTS, eu-north-1b
Security Group	launch-wizard-2	Ports 22, 8000, 8501 open to 0.0.0.0/0
IAM Role	EC2-CloudWatch-Role	CloudWatchLogsFullAccess attached to EC2 instance
CloudWatch	/solar-app/container	Log group receiving awslogs driver output

8.2 Security Group Rules

Table 11: EC2 inbound security group rules.

Port	Protocol	Source	Purpose
22	TCP	0.0.0.0/0	SSH — required for appleboy/ssh-action deployment
8000	TCP	0.0.0.0/0	FastAPI REST API
8501	TCP	0.0.0.0/0	Streamlit web user interface

9. MONITORING & OBSERVABILITY

9.1 CloudWatch Log Driver

Application logs are streamed to AWS CloudWatch using Docker's native awslogs log driver. No additional agents or SDK calls are required — the Docker daemon routes all container stdout and stderr directly to the configured log group via the --log-driver=awslogs flag at runtime.

Table 12: CloudWatch logging configuration.

Parameter	Value
Log Group	/solar-app/container
Log Stream	solar-container
Region	eu-north-1
Driver	awslogs (Docker native — no agent required)
IAM Permission	CloudWatchLogsFullAccess on EC2-CloudWatch-Role

9.2 Structured Prediction Logging

Each prediction is logged in a structured key-value format to facilitate filtering and future metric extraction in CloudWatch Insights:

```
[PREDICTION] id=42 | datetime=2024-06-15 13:30:00 | irradiation=0.75
            | amb_temp=32.5 | ac_power=3240.50 kW | mode=manual
```

Table 13: CloudWatch log filter patterns.

Filter Pattern	Returns
[PREDICTION]	All prediction events from any mode
mode=manual	User-initiated predictions via Streamlit UI
mode=auto	Scheduler-triggered predictions (Real-Time Monitor)
mode=api	Programmatic predictions via FastAPI /predict
ERROR	Application exceptions and error conditions

10. CONFIGURATION & SECURITY

10.1 Environment Variables

Table 14: Application environment variables.

Variable	Required	Default	Description
WEATHER_API_KEY	Optional	None	OpenWeatherMap key. App falls back to last CSV row then random values if absent.
SOLAR_CITY	Optional	Mumbai	City name for OpenWeatherMap weather queries

10.2 Security Practices

- All API keys and cloud credentials stored exclusively in GitHub Actions Secrets — never in source code
- Secrets injected at runtime via environment variables — never written to files or included in Docker images
- The EC2 SSH private key (.pem) is stored only as a GitHub Secret and never committed to the repository
- .gitignore excludes *.pem, .env, and model/ to prevent accidental credential and artefact exposure
- AWS IAM follows the principle of least privilege — the EC2 role grants only CloudWatch Logs permissions

11. COST ANALYSIS

Table 15: Monthly AWS service cost breakdown.

Service	Running	Stopped	Notes
EC2 t3.micro	~$0.0104/hr (~$7.50/mo)	$0.00	Primary compute cost
EBS 8 GB gp2	~$0.80/mo	~$0.80/mo	Charged regardless of EC2 state
ECR ~302 MB	~$0.03/mo	~$0.03/mo	Well within 500 MB free tier
CloudWatch	$0.00	$0.00	Within 5 GB/month free tier
IAM	$0.00	$0.00	Always free

Table 16: Cost scenarios by usage pattern.

Scenario	Estimated Monthly Cost
EC2 running 24/7	~$8.33/month
EC2 stopped (storage only)	~$0.83/month
EC2 running 8 hours/day (demo)	~$3.30/month

Remark. *Stopping the EC2 instance when not in active use reduces cost from approximately $8.33 to $0.83 per month — a reduction of approximately 90%. The ECR image and CloudWatch logs persist independently of EC2 state.*

12. LIMITATIONS & FUTURE WORK

12.1 Current Limitations

Table 17: Known limitations and proposed resolutions.

Limitation	Impact	Proposed Resolution
EC2 public IP changes on restart	EC2_HOST secret requires manual update before each deployment	Allocate an AWS Elastic IP for a permanent static address
CSV-based prediction persistence	Not suitable for concurrent writes or large-scale history	Migrate to Amazon RDS (PostgreSQL) or DynamoDB
Single container for both services	No independent scaling of UI and API workloads	Separate into two containers behind an Application Load Balancer
No TLS/HTTPS	Client-to-server traffic is unencrypted	Configure AWS Certificate Manager with an Application Load Balancer
model.pkl committed to Git	Binary artefact in version control increases repository size	Store in Amazon S3 or MLflow artefact store; pull at build time
No automated test suite in CI	Model regressions may propagate to production undetected	Implement pytest coverage for feature engineering, inference, and API responses

12.2 Proposed Enhancements

1. Implement HTTPS via AWS Application Load Balancer and AWS Certificate Manager
2. Develop a real-time model retraining trigger when rolling prediction error exceeds a defined threshold
3. Integrate Prometheus metrics endpoint and Grafana monitoring dashboard
4. Migrate prediction persistence from CSV to a managed relational database
5. Extend the forecasting horizon to 24 hours ahead using sequence models
6. Incorporate data drift detection using the Evidently AI framework

13. CONCLUSION

This report has presented the design, implementation, and deployment of a complete MLOps pipeline for solar power generation forecasting. The system integrates data versioning, experiment tracking, automated model selection, containerized multi-interface deployment, and live cloud monitoring into a unified, reproducible workflow.

The architecture embodies core MLOps principles: reproducibility through DVC, experiment governance through MLflow, environment parity through Docker, deployment automation through GitHub Actions, and observability through CloudWatch. The benchmark evaluation gate ensures that only models meeting defined quality thresholds reach production, providing a principled mechanism for model governance.

While the current implementation targets an academic context with cost-optimised infrastructure choices, the design is intentionally extensible — each component can be independently upgraded to production scale without fundamental architectural changes.

Table 18: Deliverable completion status.

Deliverable	**Status**
Data versioning with DVC	Complete
Experiment tracking with MLflow	Complete
MLflow Model Registry with versioning	Complete
Benchmark-gated model evaluation (evaluate.py)	Complete
Streamlit dashboard — three pages	Complete
FastAPI REST API — five endpoints	Complete
Docker dual-app containerization	Complete
GitHub Actions CI/CD pipeline	Complete
AWS ECR image registry	Complete
AWS EC2 cloud deployment	Complete
AWS CloudWatch structured log monitoring	Complete

— End of Report —